



Maschinelles Lernen

3. Optimierung

Prof. Dr. Dennis Janka

Ziele dieses Kapitels

- Grundbegriffe der mathematischen Optimierung
- Optimalitätsbedingungen: wann ist ein Punkt ein Minimum?
- Analytische Lösung für lineare Regression (Normalengleichung)
- Gradientenabstieg: das wichtigste Verfahren im maschinellen Lernen
- Anwendung des Gradientenabstiegs auf die lineare Regression

Gliederung

- 3.1 Grundbegriffe
- 3.2 Optimalitätsbedingungen
- 3.3 Analytische Lösung: lineare Regression
- 3.4 Gradientenabstieg (univariat)
- 3.5 Gradientenabstieg (multivariat)
- 3.6 Gradientenabstieg für lineare Regression



Grundbegriffe



Das Optimierungsproblem

Im maschinellen Lernen suchen wir immer nach **optimalen Modellparametern** — das ist ein Optimierungsproblem.

Standardform:

$$\min_{\mathbf{x} \in \mathbb{R}^n} g(\mathbf{x})$$

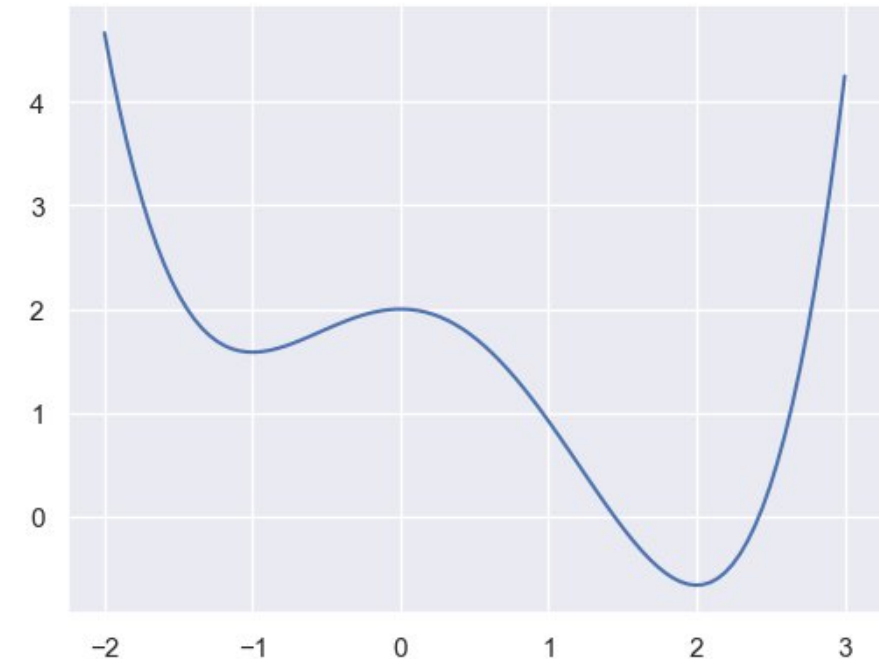
- $\mathbf{x} \in \mathbb{R}^n$: **Optimierungsvariablen** (im ML: Parameter / Gewichte $\mathbf{w}$)
- $g: \mathbb{R}^n \rightarrow \mathbb{R}$: **Zielfunktion** (im ML: Loss-Funktion L)

Wir nehmen an: g ist stetig differenzierbar, keine Einschränkungen an $\mathbf{x}$.

Lokale und globale Minima

- **Globales Minimum $x^\star$:** $g(x) \geq g(x^\star)$ für **alle** x
- **Lokales Minimum $x^\star$:** $g(x) \geq g(x^\star)$ für alle x in einer Umgebung von $x^\star$
- **Unbeschränktes Problem:** kein globales Minimum (z. B. $g(x) = x$)

Jedes globale Minimum ist auch ein lokales Minimum — aber nicht umgekehrt!



Lokales und globales Minimum



Optimalitätsbedingungen



Optimalitätsbedingungen (univariat)

Aus der Schule bekannt (Kurvendiskussion):

Sei $g:\mathbb{R} \rightarrow \mathbb{R}$ zweimal stetig differenzierbar. Dann gilt für $x_0 \in \mathbb{R}$:

- $g'(x_0) = 0$ und $g''(x_0) > 0 \Rightarrow$ lokales **Minimum**
- $g'(x_0) = 0$ und $g''(x_0) < 0 \Rightarrow$ lokales **Maximum**

Punkte mit $g'(x_0) = 0$ heißen **stationäre Punkte** (oder kritische Punkte).

Der Gradient: Verallgemeinerung auf mehrere Dimensionen



Für $g: \mathbb{R}^n \rightarrow \mathbb{R}$ ersetzt der **Gradient** ∇g die erste Ableitung:

$$\nabla g(\mathbf{x}_0) = \begin{pmatrix} \frac{\partial g}{\partial x_1}(\mathbf{x}_0) \\ \vdots \\ \frac{\partial g}{\partial x_n}(\mathbf{x}_0) \end{pmatrix} \in \mathbb{R}^n$$

Notwendige Bedingung für ein Minimum: $\nabla g(\mathbf{x}_0) = \mathbf{0}$

Das ist ein **Gleichungssystem** in n Variablen



Optimalitätsbedingungen (multivariat)

Die **Hessematrix** $\nabla^2 g$ enthält die zweiten partiellen Ableitungen:

$$\nabla^2 g(\mathbf{x}_0) = \begin{pmatrix} \frac{\partial^2 g}{\partial x_1^2} & \dots & \frac{\partial^2 g}{\partial x_1 \partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial^2 g}{\partial x_n \partial x_1} & \dots & \frac{\partial^2 g}{\partial x_n^2} \end{pmatrix} \in \mathbb{R}^{n \times n}$$

Für $\mathbf{x}_0$ mit $\nabla g(\mathbf{x}_0) = \mathbf{0}$:

- $\nabla^2 g(\mathbf{x}_0)$ **positiv definit** $\Rightarrow$ lokales **Minimum**
- $\nabla^2 g(\mathbf{x}_0)$ **negativ definit** $\Rightarrow$ lokales **Maximum**



Analytische Lösung: Lineare Regression



Training als Optimierungsproblem

Das Training der linearen Regression ist ein Optimierungsproblem:

$$\min_{\mathbf{w} \in \mathbb{R}^p} L(\mathbf{w}) = \frac{1}{N} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2$$

- Optimierungsvariablen: Koeffizientenvektor $\mathbf{w} \in \mathbb{R}^p$
- Zielfunktion: MSE-Verlustfunktion L
- Trainingsdaten $(\mathbf{X}, \mathbf{y})$ sind **fest** — nur $\mathbf{w}$ wird optimiert

Die Normalengleichung

Durch Nullsetzen des Gradienten $\nabla L(\mathbf{w}) = \mathbf{0}$ erhält man die **Normalengleichung**:

$$\mathbf{X}^T \mathbf{X} \mathbf{w} = \mathbf{X}^T \mathbf{y}$$

Falls $\mathbf{X}^T \mathbf{X}$ invertierbar ist, folgt die **analytische Lösung**:

$$\hat{\mathbf{w}} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

Bedeutung der analytischen Lösung

- Die lineare Regression ist eines der **wenigen Modelle** mit einer analytisch berechenbaren Lösung
- Vorteil: keine iterativen Verfahren nötig, exakte Lösung in einem Schritt
- Nachteil: für sehr große Featurematrizen **X** teuer (Matrixinversion kostet $O(p^3)$)
- In der Praxis: bei großem N oder p oft doch Gradientenverfahren verwendet
- Bei komplexeren Modellen (z.B. **neuronalen Netzen**) gibt es keine analytische Lösung — dort ist Gradientenabstieg unumgänglich.



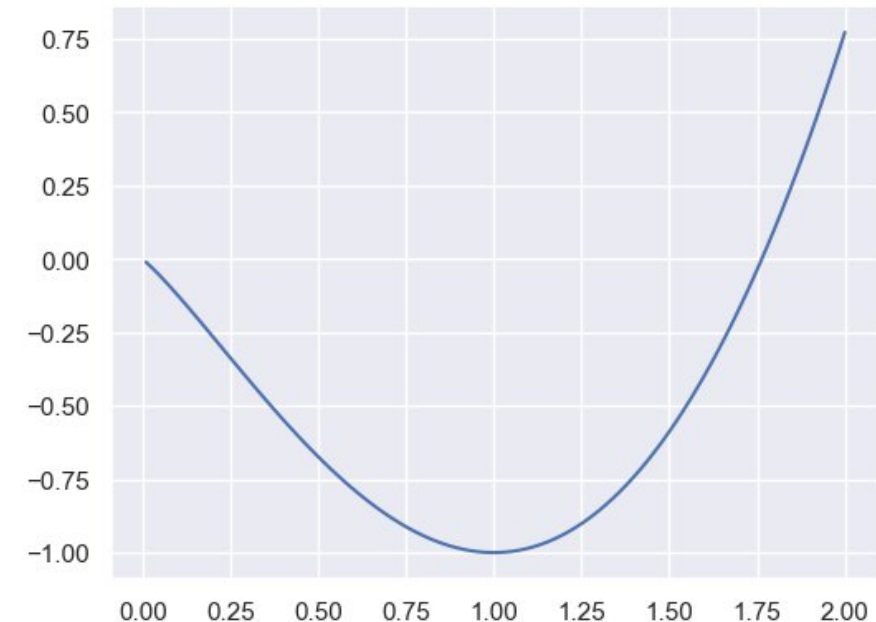
Gradientenabstieg (univariat)



Die Grundidee

Idee: Wir nähern uns einem Minimum iterativ an, indem wir uns Schritt für Schritt **bergab** bewegen.

- Wenn $g'(x) > 0$: Funktion steigt nach rechts → gehe nach **links**
- Wenn $g'(x) < 0$: Funktion fällt nach rechts → gehe nach **rechts**
- Updateformel: $x^{[k+1]} = x^{[k]} - \alpha \cdot g'(x^{[k]})$



Beispielfunktion $g(x) = x^2 \ln(x) + x - 1$

Der Algorithmus (univariat)

Gegeben: Differenzierbare Funktion $g:\mathbb{R} \rightarrow \mathbb{R}$, Startwert $x^{[0]}$, Lernrate α

Iteration: Für $k = 0, 1, 2, \dots$:

$$x^{[k+1]} = x^{[k]} - \alpha \cdot g'(x^{[k]})$$

Abbrechen, wenn eine **Abbruchbedingung** erfüllt ist.

Einfluss der Lernrate α

	$\alpha = 0.1$	$\alpha = 0.5$
Iterationen bis $ x^{[k]} - x^\star < 0.01$	15	9
Verhalten	gleichmäßig konvergent	schneller, aber unruhiger



- α **zu klein**: sehr langsame Konvergenz
- α **zu groß**: kann divergieren oder oszillieren





Gradientenabstieg (multivariat)



Geometrische Bedeutung des Gradienten

$$\nabla g(\mathbf{x}) \in \mathbb{R}^n$$

- $\nabla g(\mathbf{x})$ zeigt in Richtung des steilsten **Anstiegs**.
- $-\nabla g(\mathbf{x})$ zeigt in Richtung des steilsten **Abstiegs**
- $\| \nabla g(\mathbf{x}) \|$ gibt die Stärke des Anstiegs/Abstiegs an
- Im Minimum: $\nabla g(\mathbf{x}^\star) = \mathbf{0}$ — kein Anstieg in irgendeiner Richtung

Idee des Gradientenabstiegs: Von einem Punkt ein kleines Stück in Richtung $-\nabla g$ gehen.

Der Algorithmus (multivariat)

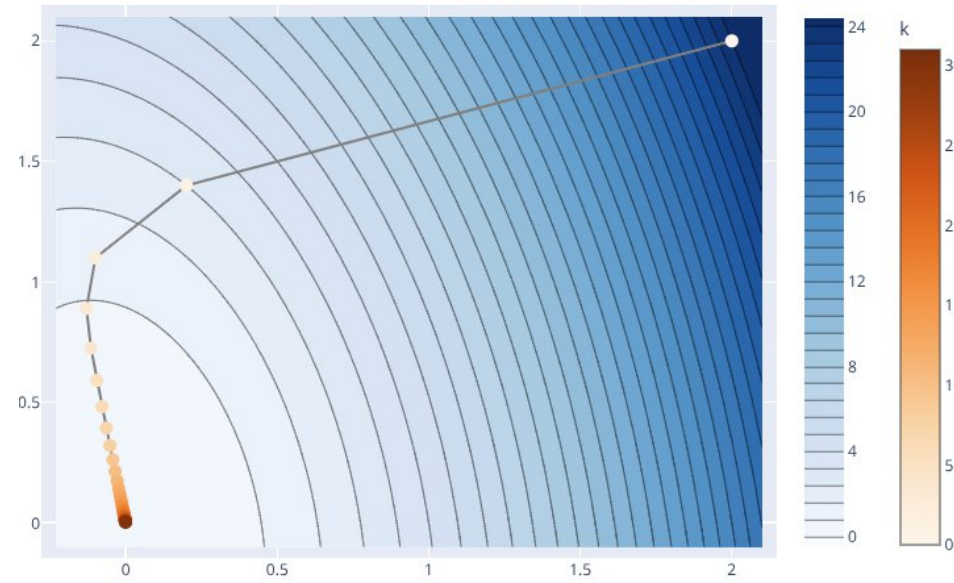
Gegeben: Differenzierbare Funktion $g: \mathbb{R}^n \rightarrow \mathbb{R}$, Startwert $\mathbf{x}^{[0]}$, Lernrate $\alpha^{[k]}$

Iteration: Für $k = 0, 1, 2, \dots$:

$$\mathbf{x}^{[k+1]} = \mathbf{x}^{[k]} - \alpha^{[k]} \nabla g(\mathbf{x}^{[k]})$$

- Aus Zahlen x werden **Vektoren** $\mathbf{x}$
- Aus der Ableitung g' wird der **Gradient** ∇g
- Die Struktur des Algorithmus bleibt identisch

Visualisierung: Iterierten im Konturplot



- Jeder Punkt = ein Vektor $x^{[k]}$
- Konturlinien = Linien gleichen Funktionswerts
- Iterierten bewegen sich von außen nach innen zum Minimum bei (0,0)

—> Siehe Jupyter Notebook **Gradient Descent Linear Regression.ipynb**

Was kann schiefgehen?

- **Divergenz:** α zu groß $\rightarrow$ Iteration springt über das Minimum hinaus und divergiert
- **Langsame Konvergenz:** α zu klein, oder schlecht konditioniertes Problem (stark unterschiedliche Krümmung in verschiedene Richtungen)
- **Lokales Minimum:** Verfahren konvergiert zu einem lokalen, nicht globalen Minimum
- **Sattelpunkt:** Gradient = 0, aber kein Minimum

Diese Fragen werden im Kontext neuronaler Netze in Kapitel 7 vertieft.



Gradientenabstieg für lineare Regression



Gradient der Loss-Funktion

Erinnerung: Wir möchten folgendes Optimierungsproblem lösen:

$$\min_{\mathbf{w} \in \mathbb{R}^p} L(\mathbf{w}) = \frac{1}{N} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2$$

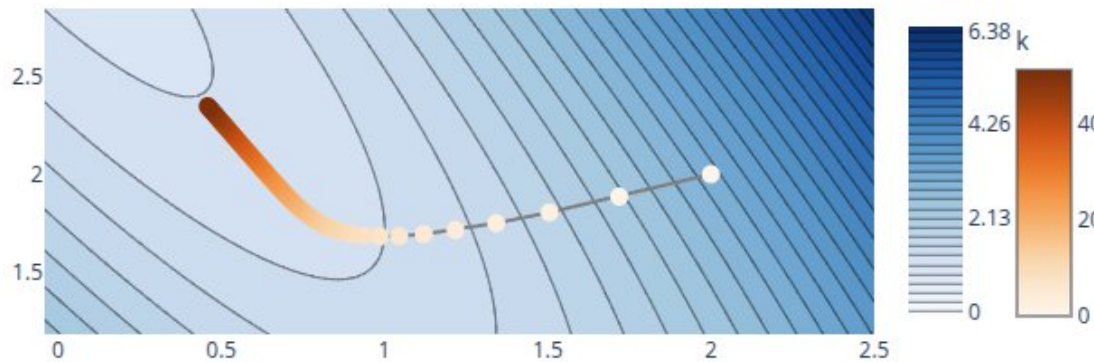
Dafür brauchen wir den Gradienten von $L(\mathbf{w})$. Dieser ist:

$$\nabla L(\mathbf{w}) = -\frac{2}{N} (\mathbf{y} - \mathbf{X}\mathbf{w})^T \mathbf{X}$$

Die Gradientenabstieg-Iteration lautet damit:

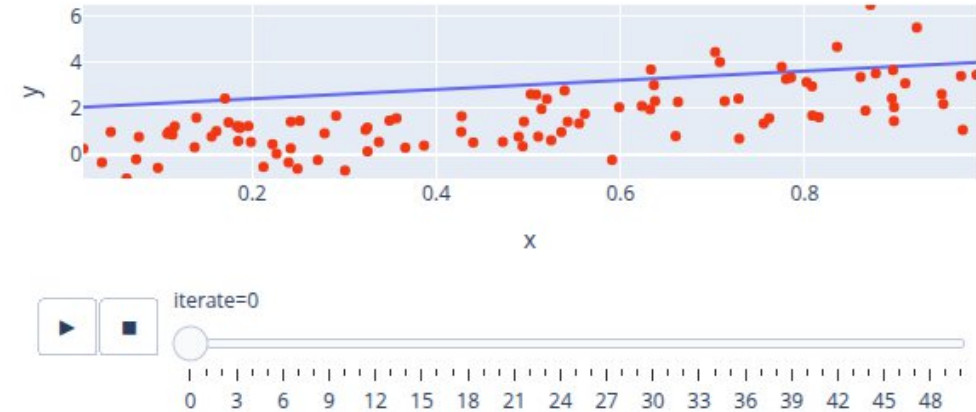
$$\mathbf{w}^{[k+1]} = \mathbf{w}^{[k]} + \frac{2\alpha^{[k]}}{N} (\mathbf{y} - \mathbf{X}\mathbf{w}^{[k]})^T \mathbf{X}$$

Gradientenabstieg in Aktion



Loss-Landschaft (Konturplot)

- Jeder Punkt = ein Paar (w_0, w_1) = eine Gerade
- Iterierten bewegen sich vom Startwert zum Minimum



Animierte Regressionsgerade

- Jede Iteration = eine Gerade im Streuplot
- Gerade bewegt sich in die Punktwolke

—> Siehe Jupyter Notebook [Gradient Descent Linear Regression.ipynb](#)



Für neuronale Netze gibt es keine analytische Lösung — Gradientenabstieg ist das Standardverfahren.



Zusammenfassung

- Modelltraining = **Optimierungsproblem**: finde $\mathbf{w}^\star = \operatorname{argmin}_{\mathbf{w}} L(\mathbf{w})$
- Optimalitätsbedingung: $\nabla L(\mathbf{w}^\star) = 0$
- Lineare Regression: analytische Lösung via **Normalengleichung** $\hat{\mathbf{w}} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$
- **Gradientenabstieg**: iterative Annäherung durch Schritte in Richtung $-\nabla L$
- Wichtige Designentscheidung: Wahl der **Lernrate** α

Weiter: Kapitel 4 — Logistische Regression